

Aircraft Engine Remaining Useful Life (RUL) Prediction

Machine Learning Project Report

Predictive Maintenance using NASA CMAPSS Turbofan Engine Data

Project Type	Machine Learning Regression + Streamlit Dashboard
Objective	Predict the Remaining Useful Life of an aircraft engine and classify health status.
Best Model	Random Forest Regressor
Final Test RMSE	31.9983
Dashboard States	Healthy, Warning, Critical

1. Executive Summary

This project develops a machine learning-based predictive maintenance system for aircraft engines. The system predicts Remaining Useful Life (RUL), which represents the estimated number of operational cycles an engine can continue before failure or major maintenance is required. The final solution includes both a trained regression model and a Streamlit dashboard that converts sensor inputs into a user-friendly health status: Healthy, Warning, or Critical.

The project uses the NASA CMAPSS turbofan engine dataset. The notebook workflow includes data loading, RUL creation, feature preparation, model training, hyperparameter tuning, evaluation, model saving, and dashboard integration.

2. Problem Statement

Aircraft engines generate multiple sensor readings during operation. These readings can indicate gradual degradation over time. Traditional maintenance approaches may either replace components too early or fail to detect risk early enough. The goal of this project is to use historical sensor data to predict engine RUL and support better maintenance decisions.

3. Project Objectives

- Load and understand the aircraft engine sensor dataset.
- Create the RUL target variable from engine cycle information.
- Train and compare multiple regression models.
- Select the best model using validation metrics.
- Save the trained model, scaler, and feature list for deployment.
- Build a Streamlit dashboard for real-time RUL prediction.
- Classify engine condition into Healthy, Warning, and Critical categories.

4. Dataset Description

The project uses NASA CMAPSS-style turbofan engine degradation data. The dataset contains training engine records, test engine records, and actual RUL values for the test engines.

File	Description	Shape Used in Notebook
train_FD001.txt	Training records containing engine cycles, operational settings, and sensor values.	20,631 rows × 26 columns
test_FD001.txt	Test records containing partial engine histories before failure.	13,096 rows × 26 columns
RUL_FD001.txt	Actual RUL values for each test engine.	100 rows × 1 column

5. Features Used

The model was trained using 24 input features. The columns `engine_id` and `cycle` were removed from the feature matrix, while RUL was used as the target variable.

Feature Group	Features
Operational Settings	op1, op2, op3
Sensor Measurements	sensor1 to sensor21
Target Variable	RUL
Dropped Columns	engine_id, cycle

6. Data Preprocessing and RUL Creation

For each engine, the maximum cycle value was calculated. RUL was then created by subtracting the current cycle from the maximum cycle of that engine.

Formula used: $RUL = \text{max_cycle} - \text{current_cycle}$

After RUL creation, the feature matrix X was prepared by removing `engine_id`, `cycle`, and RUL. The target variable y contained the RUL values.

`StandardScaler` was used to scale the feature values before training. Scaling is important because algorithms such as SVR are sensitive to different feature magnitudes.

7. Models Trained

Four regression models were trained and compared. GridSearchCV was used for hyperparameter tuning with 3-fold cross-validation and negative root mean squared error as the scoring metric.

Model	Purpose
SVR	A kernel-based regression model useful for non-linear relationships.
Random Forest Regressor	An ensemble model using multiple decision trees to improve prediction stability.
Gradient Boosting Regressor	A boosting-based model that learns from previous prediction errors.
XGBoost Regressor	A high-performance gradient boosting algorithm often used for structured datasets.

8. Model Evaluation

The models were evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score. Lower MAE and RMSE values indicate better prediction accuracy, while a higher R^2 value indicates better explanation of variance in the target variable.

Model	MAE	RMSE	R^2
Random Forest	29.3368	41.0999	0.6303
XGBoost	29.4685	41.1439	0.6295
Gradient Boosting	29.7993	41.2921	0.6268
SVR	29.2391	41.6503	0.6203

Random Forest was selected as the best model because it achieved the lowest validation RMSE among the trained models.

9. Overfitting Check

Metric	Value
Train RMSE	35.5120
Validation RMSE	41.0999

The training RMSE is lower than the validation RMSE, which is expected. The gap is present but not extremely large, suggesting that the model generalises reasonably well while still leaving scope for improvement.

10. Final Test Performance

Final Test Metric	Value
Final MAE	23.9771
Final RMSE	31.9983

The final test prediction was performed on the last available cycle of each test engine and compared with the actual RUL values from RUL_FD001.txt.

11. Streamlit Dashboard

The trained model was deployed through a Streamlit dashboard. The dashboard allows users to enter sensor values, scales the input using the saved scaler, predicts RUL using the trained Random Forest model, and displays the result using metrics and interactive Plotly charts.

Dashboard Component	Description
Sensor Inputs	Allows the user to enter operational and sensor values.
RUL Prediction	Displays the predicted Remaining Useful Life.
Health Percentage	Converts RUL into a percentage using a 0–130 capped range.
Failure Risk	Shows inverse health percentage as a simple risk indicator.
Gauge Chart	Visualises RUL across Critical, Warning, and Healthy zones.
Sensor Bar Chart	Displays the entered sensor values for easy interpretation.

12. Health Classification Logic

The dashboard uses rule-based classification after the RUL prediction is generated. This makes the final output easier to understand for users who may not be familiar with regression outputs.

Predicted RUL Range	Status	Risk Level
$RUL > 80$	Healthy	Low
$31 \leq RUL \leq 80$	Warning	Medium
$RUL \leq 30$	Critical	High

13. Issue Fixed in Streamlit App

The earlier dashboard showed only Healthy and Warning states in many cases because the Streamlit input form did not expose all 24 features used during model training. Some required features were silently filled with zero values or fixed defaults, which made the model input unrealistic and restricted the prediction range.

The corrected version supplies all features used during training, keeps the same feature order as `features.pkl`, and uses clear status logic: Critical for $RUL \leq 30$, Warning for RUL between 31 and 80, and Healthy for RUL above 80.

14. Saved Project Files

File	Purpose
<code>rul_model.pkl</code>	Saved trained Random Forest model used for prediction.
<code>scaler.pkl</code>	Saved StandardScaler used to transform input values before prediction.
<code>features.pkl</code>	Saved list of feature names and order used during training.
<code>app.py</code>	Main Streamlit dashboard file.
<code>bihaa.ipynb</code>	Notebook containing data preparation, model training, evaluation, and model saving.

15. Tools and Technologies Used

Tool / Library	Use in Project
Python	Main programming language.
Pandas	Data loading, cleaning, and transformation.
NumPy	Numerical operations and clipping predictions.
Scikit-learn	Scaling, model training, GridSearchCV, and evaluation metrics.
XGBoost	Additional boosting model for comparison.
Joblib	Saving and loading model, scaler, and feature list.
Streamlit	Interactive web dashboard.
Plotly	Gauge chart and sensor visualisation.

16. Limitations

- The model is trained on historical simulation data and may not directly generalise to real aircraft engines without domain validation.
- Only the FD001 dataset scenario is used, which represents one operating condition and one fault mode.
- Manual input of sensor values in the dashboard is useful for demonstration but real-world deployment would require live sensor integration.
- The current RUL cap of 130 is applied in the dashboard; for consistency, the training target can also be capped during notebook training.

17. Future Scope

- Train the model on all CMAPSS subsets such as FD001, FD002, FD003, and FD004.
- Apply RUL capping during model training to improve dashboard consistency.
- Add feature importance visualisation to explain which sensors influence prediction most.
- Add batch CSV upload support for predicting multiple engines at once.
- Deploy the dashboard on Streamlit Community Cloud or another hosting platform.
- Improve model performance using advanced feature engineering, rolling window statistics, and sequence-based models such as LSTM.

18. Conclusion

This project demonstrates how machine learning can support predictive maintenance by estimating the Remaining Useful Life of aircraft engines from operational and sensor data. Among the trained models, Random Forest performed best based on validation RMSE and was saved for deployment. The final Streamlit dashboard makes the model accessible by converting numerical predictions into simple health states and visual indicators. The project is a strong practical example of applying data science, regression modelling, and dashboard development to a real-world engineering maintenance problem.

Appendix A: Model Workflow Summary

1. Import required Python libraries.
2. Load train_FD001.txt, test_FD001.txt, and RUL_FD001.txt.
3. Assign column names for engine ID, cycle, operational settings, and sensors.
4. Create RUL using maximum cycle per engine.
5. Prepare X and y by removing engine_id, cycle, and RUL from the feature matrix.
6. Split training data into training and validation sets.
7. Scale features using StandardScaler.
8. Train SVR, Random Forest, Gradient Boosting, and XGBoost models.
9. Tune selected hyperparameters using GridSearchCV.
10. Evaluate models using MAE, RMSE, and R^2 .
11. Select Random Forest as the best model.
12. Save model, scaler, and feature list using Joblib.
13. Load saved files into Streamlit for real-time prediction.

Appendix B: Recommended GitHub Repository Structure

RUL-Prediction-Project/

```
|
|— app.py
|— bihaa.ipynb
|— rul_model.pkl
|— scaler.pkl
|— features.pkl
|— requirements.txt
|— README.md
|— screenshots/
|   |— dashboard_home.png
|   |— healthy_output.png
|   |— warning_output.png
|   |— critical_output.png
```